Jenny Petrova

February 10 2026

Monte Carlo and quasi-Monte Carlo Methods in Computational Finance
Chapter 1 Exercises

(3) *Given an LCG with parameters a, c, m, prove that*

$$x_{n+k} \equiv a^k x_n + \frac{(a^k - 1)}{a - 1} c \mod m; \quad (a \geq 2, k \geq 0)$$

which shows that the $(n+k)$th term can be computed directly from the nth term.

Proof. We will do a proof by induction. First, we must establish the base case. Take $k = 0$ and solve:

$$\begin{aligned} x_n &= a^0 x_n + \frac{(a^0 - 1)}{a - 1} c \mod m \\ &= (1) x_n + \frac{(1 - 1)}{a - 1} c \mod m \\ &= x_n \mod m. \end{aligned}$$

Now suppose the formula,

$$x_{n+k} \equiv a^k x_n + \frac{(a^k - 1)}{a - 1} c \mod m; \quad (a \geq 2, k \geq 0),$$

holds for some fixed k. We must show the above holds for $k + 1$. Consider the following:

$$\begin{aligned} x_{n+k+1} &= a x_{n+k} + c \mod m \\ &= a \left(a^k x_n + \frac{(a^k - 1)}{a - 1} c \right) + c \mod m \\ &= a^{k+1} x_n + \left(\frac{a(a^k - 1)}{a - 1} + 1 \right) c \mod m \\ &= a^{k+1} x_n + \left(\frac{a^{k+1} - a}{a - 1} + \frac{a - 1}{a - 1} \right) c \mod m \\ &= a^{k+1} x_n + \frac{(a^{k+1} - 1)}{a - 1} c \mod m. \end{aligned}$$

Therefore, by induction the formula holds for all $k \geq 0$. $\square$

(6) *Write a code that would implement RANDU. For debugging purposes, print x_{100} when the seed is $x_0 = 1$.*

```
# Output for randutest(100):
1673794513
```

a. *Using RANDU generate $u_1, \ldots, u_{20,002}$, where $u_n = x_n/M$. For all triplets in your sequence, (u_i, u_{i+1}, u_{i+2}), in which $0.50 \leq u_{i+1} \leq 0.51$, plot u_i versus u_{i+2}. Comment on the pattern of your scatterplot.*

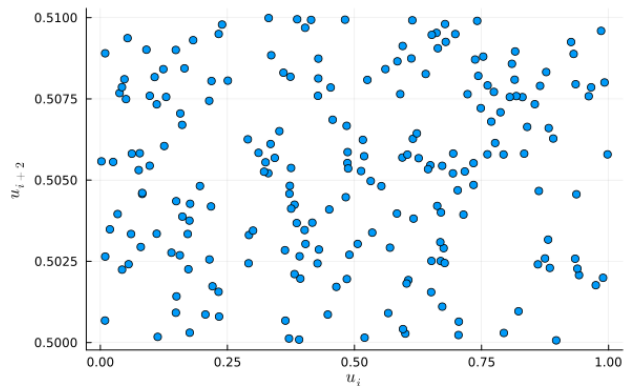

Figure 1: 2D scatterplot of u_i vs u_{i+2} for $\{u_i\}_1^{20002}$ RANDU sequence

From visual observation, the above 2D scatterplot *appears* random. However, we see the points exhibit noticeable clustering and large gaps across the plot, suggesting a potential issue of strong dependence between successive values of the RANDU generator.

b. *Generate a sequence of length* 1002. *Use a program that plots points in* 3 *dimensions and rotates the axes to rotate the points until you can see the* 15 *planes. (S-Plus function spin may be helpful.)*

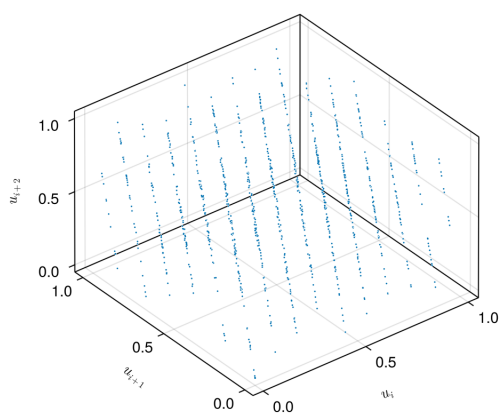

Figure 2: 3D scatterplot of u_i vs u_{i+1} vs u_{i+2} for $\{u_i\}_1^{20002}$ RANDU sequence

Examining the above 3D scatterplot, we notice something particularly suspicious: the points lie on 15 two-dimensional parallel planes. Ideally, we would expect such pseudorandomly-generated points to uniformly fill the unit cube. However, our results imply the RANDU generator does *not* produce truly random points. This supports our previous observations, where we noted the issue of strong dependence between successive outputs, hence suggesting RANDU is poor random number generator.

(8) *Prove the second inequality of Lemma 1.1 for $s = 2$.*

Lemma 1.1

For any finite set $P = \{q_1, \dots, q_N\}$, we have

$$D_N^*(P) \le D_N(P) \le 2^s D_N^*(P).$$

Proof. Consider the definition of discrepancy and star-discrepancy for $s = 2$:

$$D_N(q_n) = \sup_{[a,b)} \Big| \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{\prod_{i=1}^{2}[a_i,b_i)}(q_n) - \lambda_2([a,b)) \Big|, \quad D_N^*(q_n) = \sup_{[0,a)} \Big| \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{\prod_{i=1}^{2}[0,a_i)}(q_n) - \lambda_2([0,a)) \Big|$$

We begin with the expansion of the first term in the discrepancy. Let $q_n = (x_n, y_n) \in [0,1)^2$ and observe that

$$\begin{aligned}
\frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{\prod_{i=1}^{2}[a_i,b_i)}(q_n) &= \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[a_1,b_1)\times[a_2,b_2)}(x_n, y_n) \\
&= \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[a_1,b_1)}(x_n) \mathbf{1}_{[a_2,b_2)}(y_n) \\
&= \frac{1}{N} \sum_{n=1}^{N} \Big(\mathbf{1}_{[0,b_1)}(x_n) - \mathbf{1}_{[0,a_1)}(x_n)) (\mathbf{1}_{[0,b_2)}(y_n) - \mathbf{1}_{[0,a_2)}(y_n)) \Big) \\
&= \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,b_1)\times[0,b_2]}(x_n, y_n) - \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,a_1)\times[0,b_2]}(x_n, y_n) \\
&\quad - \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,a_2)\times[0,b_1]}(x_n, y_n) + \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,a_1)\times[0,a_2]}(x_n, y_n).
\end{aligned}$$

Now, we expand the second term in the discrepancy, such that

$$\lambda_2([a,b)) = \prod_{i=1}^{2} [a_i, b_i) = (b_1 - a_1)(b_2 - a_2) = b_1 b_2 - a_1 b_2 - b_1 a_2 - a_1 a_2.$$

We combine the above terms and apply the triangle inequality to show

$$\begin{aligned}
&\Big| \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{\prod_{i=1}^{2}[a_i,b_i)}(q_n) - \lambda_2([a,b)) \Big| \\
&= \Big| \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,b_1)\times[0,b_2]}(x_n, y_n) - \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,a_1)\times[0,b_2]}(x_n, y_n) - \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,a_2)\times[0,b_1]}(x_n, y_n) \\
&\qquad + \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,a_1)\times[0,a_2]}(x_n, y_n) - b_1 b_2 + a_1 b_2 + b_1 a_2 - a_1 a_2 \Big| \\
&\le \Big| \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,b_1)\times[0,b_2]}(x_n, y_n) - b_1 b_2 \Big| + \Big| \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,a_1)\times[0,b_2]}(x_n, y_n) - a_1 b_2 \Big| \\
&\qquad + \Big| \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,a_2)\times[0,b_1]}(x_n, y_n) - b_1 a_2 \Big| + \Big| \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}_{[0,a_1)\times[0,a_2]}(x_n, y_n) - a_1 a_2 \Big|,
\end{aligned}$$

Therefore, taking the supremum of the above result over all $0 \le a < b \le 1$ yields

$$D_N(q_n) \le 4 D_N^*(q_n) = 2^2 D_N^*(q_n).$$

□

(10) *The following is a test function commonly used in numerical integration*

$$f(x^{(1)}, \ldots, x^{(s)}) = \prod_{i=1}^{s} \frac{\pi}{2} \sin(\pi x^{(i)}).$$

The integral of f over $(0,1)^s$ is equal to 1. Estimate this integral using MC and RQMC methods by computing

$$I_k = \frac{1}{N} \sum_{n=1}^{N} f(q_n^{(1)}, \ldots, q_n^{(s)})$$

where $(q_n^{(1)}, \ldots, q_n^{(s)})^T$ is the nth term of an s-dimensional randomly shifted Halton sequence in the case of RQMC, and pseudorandom vectors generated using the Mersenne Twister in the case of MC. The subscript k of I_k refers to the kth estimate for the integral using the kth randomly shifted Halton sequence (RQMC) or Mersenne Twister (MC).

a. *Use Mersenne Twister to obtain $I_1, \ldots, I_m$ where $m = 40, N = 1000, s = 10$.*

b. *Use randomly shifted Halton sequences to obtain $I_1, \ldots, I_m$ where $m = 40, N = 1000, s = 10$.*

c. *Repeat parts (a) and (b) with $N = 10,000$.*

d. *Use box-whisker plots to display the estimates you obtained in (a) through (c) together with the exact value. Which method seems to be better in approximating the integral?*

We fix seed = 1234 for the following results.

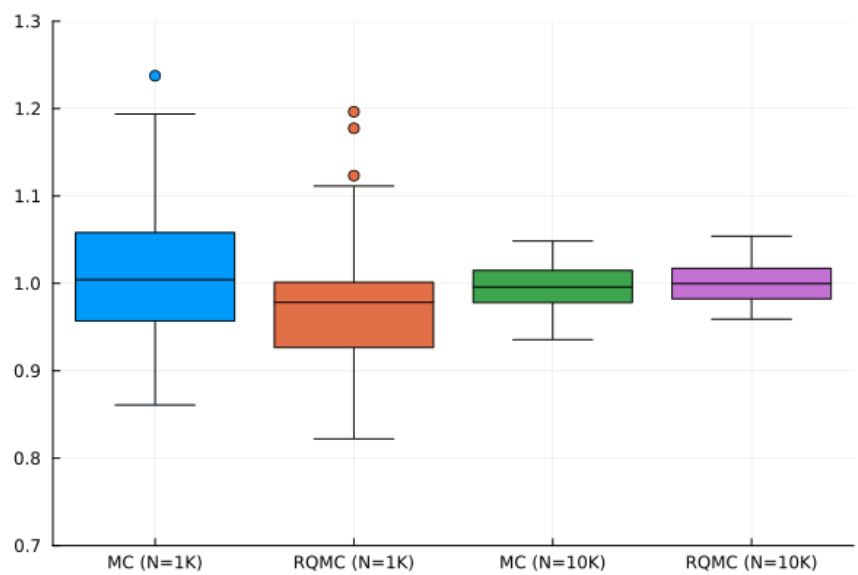

Figure 3: Box-and-whisker plots for MC vs RQMC for $m = 40$ estimates

Observing the above box-whisker plots for $m = 40$ integral estimates, $\{I_1, I_2, \ldots, I_{39}, I_{40}\}$, there is a significant improvement in the estimation, for both methods, when increasing the number of generated sequences (N). For $N = 10000$ sequences of $s = 10$ pseudorandmly-generated variables, the RQMC method produces a tighter spread than the MC method, suggesting improvements in the estimation and convergence to the true value of the integral. Notice that for $N = 1000$ and $N = 10000$, the MC method yields a median value *closer* to the true value of the integral. Despite this, the latter observation is misleading, since we want to consider estimation accuracy through the general trend in the distribution of the estimates, rather than from one single median estimate in our distribution.

Appendix

```julia
    using Random, Primes
using Pkg
using StatsPlots
using Plots
using LaTeXStrings
using GLMakie
using Plots
using GLMakie
GLMakie.activate!()

# ---------------------
# Chapter 1 Problem 6
# ---------------------

# ------- 6 (a) -------

# test function for RANDU with seed x0=1
function randutest(n,x=1)
    M = 2^(31)
    for i in 1:n
        x = mod(65539 * x, M) # RANDU generator
    end
    return x
end
x = randutest(100)
print(x)

# complete RANDU generator
function randu(n,x=1)
    seq = zeros(n)
    u_lower = []
    u_upper = []
    M = 2^(31)
    for i in 1:n
        seq[i] = x # store generated points
        x = mod(65539 * x, M) # RANDU generator
    end
    u = seq / M
    for i in 1:n
        # if 0.5 <= u_{i+1} <= 0.51, then store u_{i} and u_{i+2} as coordinate pairs
        if u[i] <= 0.51 && u[i] >= 0.50
            push!(u_lower, u[i-1])
            push!(u_upper, u[i])
        end
    end
    return u, u_lower, u_upper
end

u1, u_lower, u_upper = randu(20002)

# create 2D scatterplot of u_{i} vs u_{i+2}
Plots.scatter(u_lower, u_upper,
    xlabel = L"$u_{i}$",
    ylabel = L"$u_{i+2}$",
    title = "RANDU",
    legend = false
)

```

```julia
# ------- 6 (b) -------

u2, _, _ = randu(1002)

# store consecutive triplets
x = u2[1:1000]
y = u2[2:1001]
z = u2[3:1002]

# create 3D (interactive) scatterplot of triplets (u_{i} vs u_{i+1} vs u_{i+2})
fig = Figure()
ax = Axis3(fig[1, 1], xlabel = L"$u_i$", ylabel = L"$u_{i+1}$", zlabel = L"$u_{i+2}$")
GLMakie.scatter!(ax, x, y, z; markersize = 2)
display(fig)

# ---------------------
# Chapter 1 Problem 10
# ---------------------

m = 40
N1 = 1000
N2 = 10000
s = 10

# create generator for MC method
rng = MersenneTwister(1234)

# create generator for RQMC method
lprimes=primes(2,200)

# generate nth element of the van der Corput sequence in base b
function vdc(n,b)
    digs=digits(n,base=b)
    sum=0.0
    for i in 1:length(digs)
        sum = sum + digs[i]/b^i
    end
    return(sum)
end

# get the nth element of the s-dimensional Halton sequence
function halton(n,s)
    [vdc(n,i) for i in lprimes[1:s]]
end

# randomly shift the s-dimensional Halton sequence
function shifthalton(n,s)
    u = rand(rng, s)
    map(k->mod(k,1),halton(n,s).+u)
end

# estimate the integral for Nxs matrix with N pseudorandomly generated s-dimensional sequences
function fintegral(N, s, method="rqmc")
    I = 0.0
    for i in 1:N
        if method=="mc"
            q = rand(rng, s)
        else
            q = shifthalton(i, s)
        end
```

```
        term = 1.0
        for j in 1:(length(q))
            term *= (pi/2) * sin(pi * q[j])
        end
        I += term
    end
    return (I/N)
end

# print integral estimates for MC and RQMC, for N=1000 and N=10,000
println("Integral Estimates for N=$N1")
I_mc1 = [fintegral(N1, s, "mc") for _ in 1:m]
println("Mersenne Twister: $I_mc1")
I_rqmc1 = [fintegral(N1, s) for _ in 1:m]
println("Randomly shifted Halton: $I_rqmc1")
println("Integral Estimates for N=$N2")
I_mc2 = [fintegral(N2, s, "mc") for _ in 1:m]
println("Mersenne Twister: $I_mc2")
I_rqmc2 = [fintegral(N2, s) for _ in 1:m]
println("Randomly shifted Halton: $I_rqmc2")

# create box-and-whisker plots for each set of integral estimates
data=[I_mc1, I_rqmc1, I_mc2, I_rqmc2]
Plots.boxplot(data, legend=false)
Plots.xticks!(1:4, ["MC (N=1K)", "RQMC (N=1K)", "MC (N=10K)", "RQMC (N=10K)"])
Plots.ylims!(0.7,1.3)
```

```
# OUTPUTS
x_100 for seed x_0 = 1: 1673794513

Integral Estimates for N=1000
Mersenne Twister: [1.0058202890799817, 0.9769243020628176, 1.0570478031767925, ... ,
↪  0.9527564458898824, 0.8422999154302471, 0.982975562154701]
Randomly shifted Halton: [1.0193026427777465, 0.8838394069112271, 0.9818815387351807, ... ,
↪  0.9485128515993083, 0.863356384226958, 0.9551964029770618]

Integral Estimates for N=10000
Mersenne Twister: [0.9782856665256597, 0.9900870158560825, 0.9866250138358466, ... ,
↪  1.0119412909144712, 0.990465861619219, 0.9478905442609182]
Randomly shifted Halton: [1.0168483207214152, 1.0217322205223915, 1.0140601481747953, ... ,
↪  1.0397986894956894, 1.0125711653268537, 1.006734289853648]
```